

Themison Clinical Trial Platform - Development Summary

This document provides a comprehensive log of all achievements, structural changes, database migrations, and UI/UX enhancements made to the **Themison Clinical Trial Platform** (covering both the Frontend and Backend repositories) from inception to completion.

1. Executive Summary

The development work focused on transitioning the Themison platform from a prototype with mock components into a robust, integrated enterprise clinical SaaS application. Key achievements include:

- **Database Consolidation:** Full transition from MySQL to PostgreSQL.
 - **Advanced AI RAG (Retrieval-Augmented Generation):** Integration of Google File Search API, PDF text extraction, and bounding-box highlighting.
 - **Study Setup Wizard:** A redesigned, clinical-grade multi-step interface for operationalizing trial documents.
 - **Task & Dependency Management:** A complete full-stack integration connecting the interactive Kanban/Gantt board to a relational database schema.
 - **Patient Dashboard:** Visual layout and scheduling dashboard integration for clinical coordinators.
-

2. Detailed Technical Breakdown

2.1. Task & Phase Management System Integration

We successfully bridged the 4,000+ line interactive frontend task board to the backend PostgreSQL database using a robust relational schema.

- **Schema Upgrades (Alembic Migration):** Created the migration script `0b36ae7e7c90_add_task_manager_fields_and_dependencies.py` to add new fields to the tasks table and create a new relationship table:
 - `phase_id`: Links tasks to trial phases/visits (Screening, Treatment, Follow-up, Closeout).
 - `assigned_role`: Supports role-based task delegation (PI, CRC, Nurse, Monitor).
 - `blocked_reason / blocked_since`: Supports structured JSON metadata for task blockers.
 - `order_in_phase`: Enables drag-and-drop order persistence.
 - `suggested_date / suggested_assignee`: Added fields for AI-driven task scheduling.
 - **Task Dependencies (task_dependencies Table):** Implemented a schema to store task relationships, supporting `finish_to_start` types, cross-phase dependencies, and conditional labels.
 - **Proxy Integration (server/mapRouter.ts):** Replaced static mock endpoints with live tRPC database proxy queries, mapping backend PostgreSQL models directly to frontend typescript interfaces, ensuring creation, updates, moves, and dependency changes persist in real-time.
 - **UI Alignment:** Configured sorting on tasks by descending priority and matched page background styles with the global trial workspace theme.
-

3. Study Setup Wizard (UI/UX Redesign)

The Study Setup Wizard was completely redesigned to replace flashy, consumer-style aesthetics with a professional, clinical medical interface.

- **Three-Step Sequential Interface:**

1. **Upload Documents:** Upload protocol and auxiliary document checklist.

2. **Review Coverage:** Analyze plan completeness using an AI-driven coverage estimate.

3. **Generate Plan:** Preview extracted tasks and confirm generation.

● **Multi-Document Collection Checklist:**

- Includes **Required** sections (Protocol document cards with page counts).

- Includes **Recommended** auxiliary sections (Monitoring Plan, Safety Reporting Manual, EDC/CRF Completion Guide).

- Implemented dynamic [+ Add] buttons linked to the Document Hub upload modal and a [Add Other Document Type] custom file type generator.

- **UX Spacing & Scrolling Polish:**

- Fixed the Wizard header and navigation footer at the top and bottom of the viewport so controls remain interactive.

- Restricted scroll behavior to the middle step-content container.

- Replaced wireframe steps with Centered Arrow SVG icons and color-coded status badges (light green background #edfcf2 and checked icons #62D686 for completed steps; gray circles for upcoming steps).

- Matched the Plan Coverage progress bar colors to the global Trial Workspace progression colors.

4. AI RAG & Document Intelligence Integration

Developed an end-to-end semantic document indexing and query response system using Google Gemini and custom PDF parsers.

- **Google File Search Integration:**

- Configured backend routers (`server/_core/fileSearch.ts` and `server/documentAIRouter.ts`) to manage trial-isolated vector stores.
- Implemented auto-upload rules: newly uploaded trial documents are automatically sent to Google's vector DB.

- **Gemini Chat Session Management:**

- Integrated historical chat context so the LLM retains thread history (`client/src/pages/DocumentAIAssistant.tsx`).
- Modified the system prompt to ground responses strictly in indexed documents.

- **PDF Highlight & Bounding-Box Fallback Rendering:**

- Created local PDF text extraction via `pdf-parse v2`.
- Added a polyfill for `DOMMatrix` to solve standard PDF parsing limits under Node 20+.
- Programmed coordinate translation logic to map document citations into highlights.
- Implemented a local sentence-matching keyword highlight search fallback in Python (`PDFHighlightService` using `PyMuPDF/fitz`) for cases where coordinate bounding boxes are missing or the external RAG service is unreachable.
- Made Redis caching operations fail-safe and implemented self-healing URL translations to correct `localhost / 127.0.0.1` download path issues.

5. Layouts, Themes & Quality of Life Enhancements

- **Trial Navigation Redesign:** Replaced the legacy vertical left sidebar menu in trial details with a clean, horizontal top tab bar navigation.
- **Document Hub UI Polish:** Re-aligned container margins to match tab borders. Enclosed the Documents Title and document count on a single row next to the upload button inside the white page body.
- **Dark Mode Toggle:** Integrated a theme switcher into the top navigation header.
- **Robust Navigation Routing:** Resolved errors caused by trial ID format conflicts (e.g., NaN slug errors when converting between numeric database IDs and alphanumeric trial slugs) by standardizing trial IDs as strings across all backend routers and database schemas.

6. Seeding & Testing Scripts

- **Vitest Integration:** Configured tests to assert backend chat routing endpoints.
- **Clinical Demo Seed Data (`seed-wizard-demo.mjs` & `seed-trials.mjs`):** Scripted database seeds that insert realistic clinical trials, phases, pre-trial tasks, and sample patient lists.
- **Loading State Polish:** Created a dynamic loading indicator that displays changing "thinking" statements every 2.5 seconds to represent long-running AI operations.

7. Core Directory & File Mapping

Below is a map of the main codebases where modifications took place:

Frontend Workspace (`Demo-New-Frontend-4`)

- `client/src/pages/Tasks.tsx` — Full-featured Kanban/Gantt task coordinator.

- `client/src/pages/DocumentAIAssistant.tsx` — AI Chat Assistant with dropdown selectors.

- `client/src/pages/TrialDetail.tsx` — Central Trial detail view with horizontal tabs.

- `server/mapRouter.ts` — tRPC backend proxy for mapping PostgreSQL tasks.

- `server/documentAIRouter.ts` — Gemini LLM & Google File Search RAG route.

- `server/documentsRouter.ts` — File processing and upload flow.

- `seed-wizard-demo.mjs` — Local database seeding utility.

Backend Workspace (Demo - New - Backend)

- `app/models/tasks.py` — Database models for Tasks.
 - `app/contracts/tasks.py` — Pydantic schemas for data validation.
 - `app/api/routes/api/tasks.py` — API REST routes for task mutations.
 - `alembic/versions/20260601_0858_*.py` — Alembic database migration blueprint.
 - `app/main.py` — Main entry point for FastAPI routes and middle-tier proxies.
 - `scratch/rag-service/.../highlighting_service.py` — Python-based PyMuPDF PDF highlight compiler.
-

8. Current Status & Verification

- **Database Migrations:** Applied successfully.
- **FastAPI & Express Proxy:** Connected and routing requests without CORS errors.
- **Unit Tests:** Vitest runs are clean; authentication fallbacks successfully capture mock states during local offline development.